JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Spliterators.AbstractDoubleSpliterator

java.lang.Object
  java.util.Spliterators.AbstractDoubleSpliterator

All Implemented Interfaces:

Spliterator<Double>, Spliterator.OfDouble, Spliterator.OfPrimitive<Double, DoubleConsumer, Spliterator.OfDouble>

Enclosing class:

Spliterators

```
public abstract static class Spliterators.AbstractDoubleSpliterator
extends Object
implements Spliterator.OfDouble
```

An abstract `Spliterator.OfDouble` that implements `trySplit` to permit limited parallelism.

To implement a spliterator an extending class need only implement `tryAdvance`. The extending class should override `forEachRemaining` if it can provide a more performant implementation.

API Note:

This class is a useful aid for creating a spliterator when it is not possible or difficult to efficiently partition elements in a manner allowing balanced parallel computation.

An alternative to using this class, that also permits limited parallelism, is to create a spliterator from an iterator (see `Spliterators.spliterator(java.util.PrimitiveIterator.OfDouble, long, int)`. Depending on the circumstances using an iterator may be easier or more convenient than extending this class. For example, if there is already an iterator available to use then there is no need to extend this class.

Since:

1.8

See Also:

`Spliterators.spliterator(java.util.PrimitiveIterator.OfDouble, long, int)`

Nested Class Summary

Nested classes/interfaces declared in interface Spliterator
`Spliterator.OfDouble`, `Spliterator.OfInt`, `Spliterator.OfLong`, `Spliterator.OfPrimitive<T,T_CONS,T_SPLITR>`

Field Summary

Fields declared in interface Spliterator
`CONCURRENT`, `DISTINCT`, `IMMUTABLE`, `NONNULL`, `ORDERED`, `SIZED`, `SORTED`, `SUBSIZED`

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	`AbstractDoubleSpliterato (long est, int additionalCharacteri`	Creates a spliterator reporting the given estimated size and characteristics.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
int	`characteristics()`	Returns a set of characteristics of this Spliterator and its elements.
long	`estimateSize()`	Returns an estimate of the number of elements that would be encountered by a `Spliterator.forEachRemaining(Consumer)`

	traversal, or returns `Long.MAX_VALUE` if infinite, unknown, or too expensive to compute.
`Spliterator.OfDouble` **`trySplit`**`()`	If this spliterator can be partitioned, returns a Spliterator covering elements, that will, upon return from this method, not be covered by this Spliterator.

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Methods declared in interface Spliterator

`getComparator`, `getExactSizeIfKnown`, `hasCharacteristics`

Methods declared in interface Spliterator.OfDouble

`forEachRemaining`, `tryAdvance`

Methods declared in interface Spliterator.OfPrimitive

`forEachRemaining`, `tryAdvance`

Constructor Details

AbstractDoubleSpliterator

```
protected AbstractDoubleSpliterator(long est,
                                    int additionalCharacteristics)
```

Creates a spliterator reporting the given estimated size and characteristics.

Parameters:

`est` - the estimated size of this spliterator if known, otherwise `Long.MAX_VALUE`.

`additionalCharacteristics` - properties of this spliterator's source or elements. If `SIZED` is reported then this spliterator will additionally report `SUBSIZED`.

Method Details

trySplit

`public Spliterator.OfDouble trySplit()`

If this spliterator can be partitioned, returns a Spliterator covering elements, that will, upon return from this method, not be covered by this Spliterator.

If this Spliterator is `Spliterator.ORDERED`, the returned Spliterator must cover a strict prefix of the elements.

Unless this Spliterator covers an infinite number of elements, repeated calls to `trySplit()` must eventually return `null`. Upon non-null return:

- the value reported for `estimateSize()` before splitting, must, after splitting, be greater than or equal to `estimateSize()` for this and the returned Spliterator; and
- if this Spliterator is `SUBSIZED`, then `estimateSize()` for this spliterator before splitting must be equal to the sum of `estimateSize()` for this and the returned Spliterator after splitting.

This method may return `null` for any reason, including emptiness, inability to split after traversal has commenced, data structure constraints, and efficiency considerations. This implementation permits limited parallelism.

Specified by:

`trySplit` in interface `Spliterator<Double>`

Returns:

a `Spliterator` covering some portion of the elements, or `null` if this spliterator cannot be split

estimateSize

`public long estimateSize()`

Returns an estimate of the number of elements that would be encountered by a `Spliterator.forEachRemaining(Consumer)` traversal, or returns `Long.MAX_VALUE` if infinite, unknown, or too expensive to compute.

If this Spliterator is `Spliterator.SIZED` and has not yet been partially traversed or split, or this Spliterator is `Spliterator.SUBSIZED` and has not yet been partially traversed, this estimate must be an accurate count of elements that would be encountered by a complete traversal. Otherwise, this estimate may be arbitrarily inaccurate, but must decrease as specified across invocations of `Spliterator.trySplit()`.

Specified by:

`estimateSize` in interface `Spliterator<Double>`

Implementation Requirements:

This implementation returns the estimated size as reported when created and, if the estimate size is known, decreases in size when split.

Returns:

the estimated size, or `Long.MAX_VALUE` if infinite, unknown, or too expensive to compute.

characteristics

```
public int characteristics()
```

Returns a set of characteristics of this Spliterator and its elements. The result is represented as ORed values from `Spliterator.ORDERED`, `Spliterator.DISTINCT`, `Spliterator.SORTED`, `Spliterator.SIZED`, `Spliterator.NONNULL`, `Spliterator.IMMUTABLE`, `Spliterator.CONCURRENT`, `Spliterator.SUBSIZED`. Repeated calls to `characteristics()` on a given spliterator, prior to or in-between calls to `trySplit`, should always return the same result.

If a Spliterator reports an inconsistent set of characteristics (either those returned from a single invocation or across multiple invocations), no guarantees can be made about any computation using this Spliterator.

Specified by:

`characteristics` in interface `Spliterator<Double>`

Implementation Requirements:

This implementation returns the characteristics as reported when created.

Returns:

a representation of characteristics

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other

countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.